

Library Management API Documentation

API Module Name: Authentication

Base URLs

- **Development:** http://localhost:5000/api
- **Production:** None

Endpoints Overview

Endpoint	Method	Description
/api/auth/register	POST	Register a new user account
/api/auth/login	POST	Authenticate user and receive JWT

1. User Registration

Endpoint: /api/auth/register **Method:** POST **Description:** Creates a new user profile in the system.

Request Parameters

Field Name	Type	Required	Description	Allowed Values
name	string	Yes	Full name of the user	N/A
email	string	Yes	Valid email address	N/A
password	string	Yes	Secure password	N/A

Request Body Example

```
JSON
{
  "name": "Atul",
```

```
"email": "atul@mail.com",  
"password": "atul1234"  
}
```

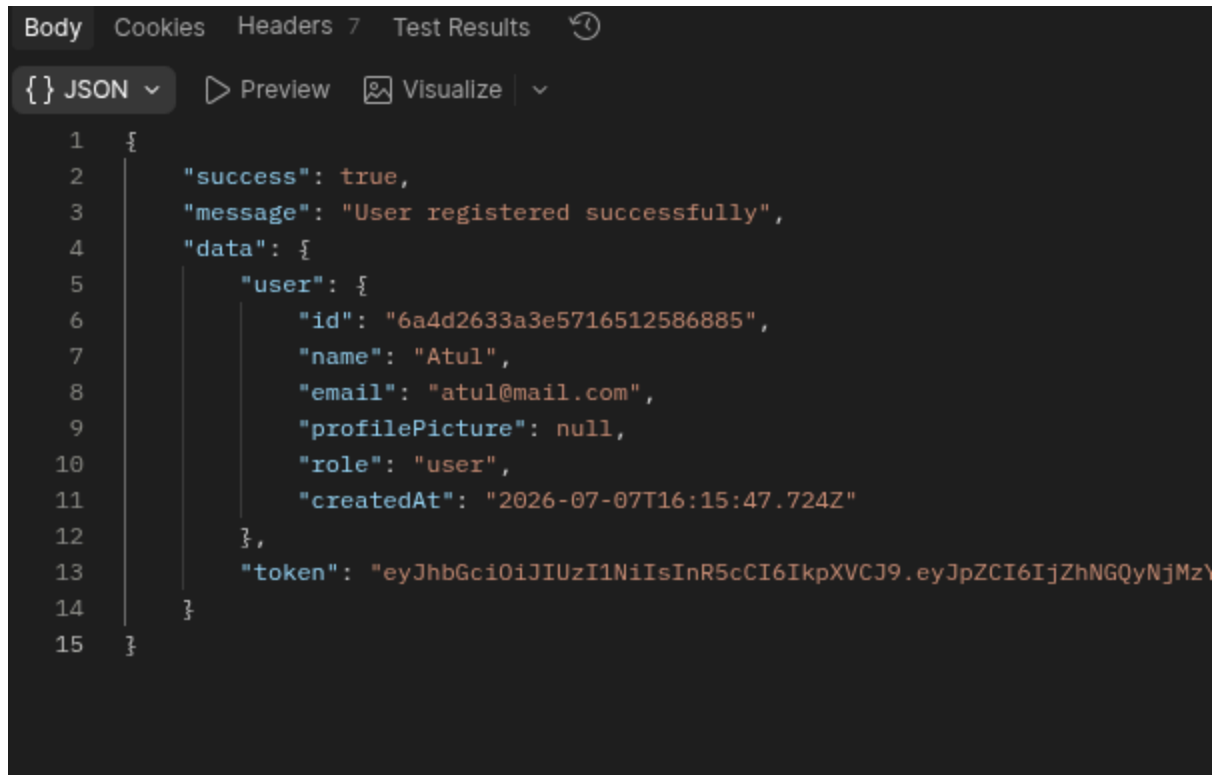
Response Statuses

Status Code	Message	Description
201 Created	User registered	Account created successfully
400 Bad Request	Invalid data	Missing fields
409 Conflict	User exists	Email already registered

Response Example

JSON

```
{  
  "success": true,  
  "message": "User registered successfully",  
  "data": {  
    "user": {  
      "id": "6a4d2633a3e5716512586885",  
      "name": "Atul",  
      "email": "atul@mail.com",  
      "profilePicture": null,  
      "role": "user",  
      "createdAt": "2026-07-07T16:15:47.724Z"  
    },  
    "token":  
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhNGQyNjMzYTNlNTcxNjUxMjU4Njg4NSIsIm1hdCI6MTc0MzQ0MDk0NywiZXhwIjozNzg0MDQ1NzQ3fQ.Z1fYvfQec_IrXm5uH82u1Q3WBW-PMQJYCdi0Ru0zU4k"  
  }  
}
```



```
1  {
2    "success": true,
3    "message": "User registered successfully",
4    "data": {
5      "user": {
6        "id": "6a4d2633a3e5716512586885",
7        "name": "Atul",
8        "email": "atul@mail.com",
9        "profilePicture": null,
10       "role": "user",
11       "createdAt": "2026-07-07T16:15:47.724Z"
12     },
13     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhNGQyNjMzY"
14   }
15 }
```

2. User Login

API Endpoint: /api/auth/login **Method:** POST **Description:** Authenticates user credentials and returns a JWT token.

Request Parameters

Field Name	Type	Required	Description
email	string	Yes	Registered email address
password	string	Yes	User password

Request Body Example

```
JSON
{
  "email": "atul@mail.com",
  "password": "atul1234"
}
```

Response Statuses

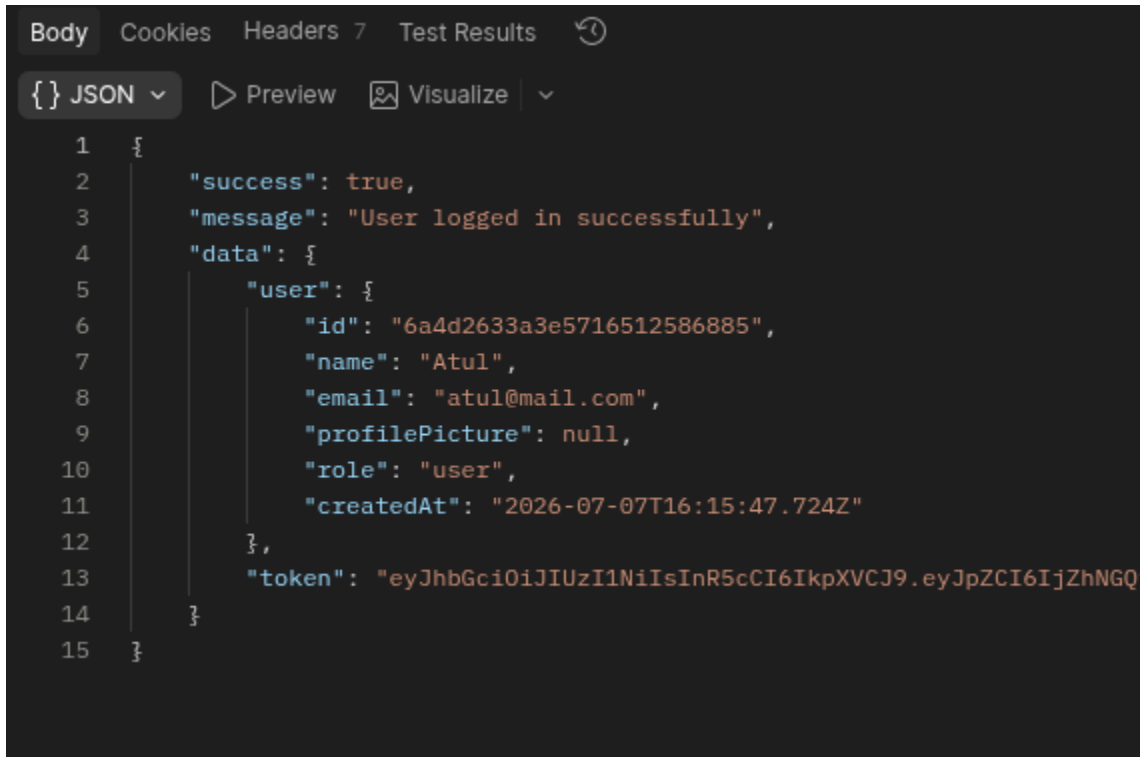
Status Code	Message	Description
200 OK	User logged in	Authentication successful
401 Unauthorized	Invalid credentials	Wrong email or password

Response Example

JSON

```
{
  "success": true,
  "message": "User logged in successfully",
  "data": {
    "user": {
      "id": "6a4d2633a3e5716512586885",
      "name": "Atul",
      "email": "atul@mail.com",
      "profilePicture": null,
      "role": "user",
      "createdAt": "2026-07-07T16:15:47.724Z"
    },
    "token":
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhNGQyNjMzYTNlNTcxNjUxMjU4Njg4NSIsIm1hdCI6MTc4MzQ0MTA2MSwiZXhwIjozNzg0MDQ1ODYxfQ.dJjhEdhoolgFOJ10tyWLRlcp1v4b-Xx-8NW8fDyjuU8"
  }
}
```

Authentication Requirement:



API Module Name: Books

Endpoints Overview

Endpoint	Method	Description
/api/books	GET	Retrieve all books (paginated, filtered)
/api/books/:id	GET	Retrieve a book by ID
/api/books	POST	Create a new book
/api/books/:id	PUT	Update book details
/api/books/:id	DELETE	Remove a book
/api/books/:id/upload-cover	PATCH	Upload book cover image

1. Get All Books

API Endpoint: /api/books **Method:** GET **Description:** Retrieves a paginated list of books with optional filters.

Query Parameters

Parameter	Type	Required	Description
page	number	No	Page number (default: 1)
limit	number	No	Items per page (default: 10)
category	string	No	Filter by exact category
author	string	No	Filter by author ObjectId
title	string	No	Search by title (case-insensitive)
publishedYear	number	No	Filter by exact year
minYear	number	No	Filter by minimum year
maxYear	number	No	Filter by maximum year

Response Example

JSON

```
{
  "success": true,
  "message": "Books retrieved successfully",
  "data": [
    {
      "_id": "6a4d2857a3e5716512586887",
      "title": "The Great Gatsby",
      "author": {
        "_id": "6a4d275aa3e5716512586886",
        "name": "F. Scott Fitzgerald"
      },
      "category": "Fiction",
      "isbn": "9780743273565",
      "description": "A classic American novel",
      "publishedYear": 1925,
    }
  ]
}
```

```
        "totalCopies": 5,  
        "availableCopies": 5,  
        "coverImage": "",  
        "createdAt": "2026-07-07T16:24:55.536Z",  
        "updatedAt": "2026-07-07T16:24:55.536Z",  
        "__v": 0  
    },  
    ],  
    "pagination": {  
        "page": 1,  
        "limit": 10,  
        "total": 1,  
        "totalPages": 1  
    }  
}
```

Authentication Requirement: None **Pagination and Filters:** page, limit, category, author, title, minYear, maxYear

```
1  {
2    "success": true,
3    "message": "Books retrieved successfully",
4    "data": [
5      {
6        "_id": "6a4d2857a3e5716512586887",
7        "title": "The Great Gatsby",
8        "author": {
9          "_id": "6a4d275aa3e5716512586886",
10         "name": "F. Scott Fitzgerald"
11       },
12       "category": "Fiction",
13       "isbn": "9780743273565",
14       "description": "A classic American novel",
15       "publishedYear": 1925,
16       "totalCopies": 5,
17       "availableCopies": 5,
18       "coverImage": "",
19       "createdAt": "2026-07-07T16:24:55.536Z",
20       "updatedAt": "2026-07-07T16:24:55.536Z",
21       "__v": 0
22     },
23   ],
24   "pagination": {
25     "page": 1,
26     "limit": 10,
27     "total": 1,
28     "totalPages": 1
29   }
30 }
```

2. Get Book by ID

API Endpoint: /api/books/:id **Method:** GET **Description:** Retrieves a single book by its ObjectId.

URL Parameters

Parameter	Type	Required	Description
id	string	Yes	Book ObjectId

Response Statuses

Status Code	Message	Description
200 OK	Book retrieved	Book found
404 Not Found	Book not found	Invalid ID

Authentication Requirement: None

```
1  {
2    "success": true,
3    "message": "Book retrieved successfully",
4    "data": {
5      "_id": "6a4d2857a3e5716512586887",
6      "title": "The Great Gatsby",
7      "author": {
8        "_id": "6a4d275aa3e5716512586886",
9        "name": "F. Scott Fitzgerald"
10     },
11     "category": "Fiction",
12     "isbn": "9780743273565",
13     "description": "A classic American novel",
14     "publishedYear": 1925,
15     "totalCopies": 5,
16     "availableCopies": 5,
17     "coverImage": "",
18     "createdAt": "2026-07-07T16:24:55.536Z",
19     "updatedAt": "2026-07-07T16:24:55.536Z",
20     "__v": 0
21   }
22 }
```

3. Create Book

API Endpoint: /api/books **Method:** POST **Description:** Adds a new book to the library.

Request Parameters

Field Name	Type	Required	Description
title	string	Yes	Book title
author	string	Yes	Author ObjectId
category	string	Yes	Book category
isbn	string	Yes	Unique ISBN

Field Name	Type	Required	Description
description	string	No	Book description
publishedYear	number	No	Year published
totalCopies	number	Yes	Total copies (min: 0)
availableCopies	number	Yes	Available copies (min: 0)

Request Body Example

```
JSON
{
  "title": "The Great Gatsby",
  "author": "{{authorId}}",
  "category": "Fiction",
  "isbn": "9780743273565",
  "description": "A classic American novel",
  "publishedYear": 1925,
  "totalCopies": 5,
  "availableCopies": 5
}
```

Response Statuses

Status Code	Message	Description
201 Created	Book created	Book added successfully
400 Bad Request	Missing fields	Validation failed
409 Conflict	Book exists	Duplicate ISBN

Authentication Requirement: Required (JWT)

```
{ } JSON  ▾  ▶ Preview  🖼 Visualize  ▾  
1  {  
2    "success": true,  
3    "message": "Book created successfully",  
4    "data": {  
5      "title": "The Great Gatsby",  
6      "author": "6a4d275aa3e5716512586886",  
7      "category": "Fiction",  
8      "isbn": "9780743273565",  
9      "description": "A classic American novel",  
10     "publishedYear": 1925,  
11     "totalCopies": 5,  
12     "availableCopies": 5,  
13     "coverImage": "",  
14     "_id": "6a4d2857a3e5716512586887",  
15     "createdAt": "2026-07-07T16:24:55.536Z",  
16     "updatedAt": "2026-07-07T16:24:55.536Z",  
17     "__v": 0  
18   }  
19 }
```

4. Update Book

API Endpoint: /api/books/:id **Method:** PUT **Description:** Updates book details by ID.

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Book updated successfully",
4    "data": {
5      "_id": "6a4d2857a3e5716512586887",
6      "title": "The Great Gatsby (Updated)",
7      "author": {
8        "_id": "6a4d275aa3e5716512586886",
9        "name": "F. Scott Fitzgerald"
10     },
11     "category": "Fiction",
12     "isbn": "9780743273565",
13     "description": "A classic American novel",
14     "publishedYear": 1925,
15     "totalCopies": 10,
16     "availableCopies": 5,
17     "coverImage": "",
18     "createdAt": "2026-07-07T16:24:55.536Z",
19     "updatedAt": "2026-07-07T16:28:09.968Z",
20     "__v": 0
21   }
22 }
```

5. Delete Book

API Endpoint: /api/books/:id **Method:** DELETE **Description:** Removes a book from the library.

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Book deleted successfully"
4  }
```

6. Upload Book Cover

API Endpoint: /api/books/:id/upload-cover **Method:** PATCH **Description:** Uploads a cover image for a book.

Request Parameters

Field Name	Type	Required	Description
coverImage	file	Yes	Image file (jpeg, png, gif, webp)

Request Body Example

None

```
Content-Type: multipart/form-data
coverImage: book-cover.jpeg
```

Response Statuses

Status Code	Message	Description
200 OK	Cover uploaded	File saved successfully
400 Bad Request	No file / Invalid type	Upload failed

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Book cover uploaded successfully",
4    "data": {
5      "_id": "6a4d2d70ffad565265c4245",
6      "title": "The Great Gatsby",
7      "author": "6a4d275aa3e5716512586886",
8      "category": "Fiction",
9      "isbn": "9780743273565",
10     "description": "A classic American novel",
11     "publishedYear": 1925,
12     "totalCopies": 5,
13     "availableCopies": 5,
14     "coverImage": "uploads/1783442824457.jpeg",
15     "createdAt": "2026-07-07T16:46:40.741Z",
16     "updatedAt": "2026-07-07T16:47:04.526Z",
17     "__v": 0
18   }
19 }
```

API Module Name: Authors

Endpoints Overview

Endpoint	Method	Description
/api/authors	GET	Retrieve all authors
/api/authors/:id	GET	Retrieve author by ID
/api/authors	POST	Create new author
/api/authors/:id	PUT	Update author
/api/authors/:id	DELETE	Delete author

1. Get All Authors

API Endpoint: /api/authors **Method:** GET **Description:** Retrieves a paginated list of authors.

Query Parameters

Parameter	Type	Description
page	number	Page number (default: 1)
limit	number	Items per page (default: 10)
name	string	Search by name (regex)
country	string	Filter by exact country

Pagination and Filters: page, limit, name, country **Authentication Requirement:** None

```
{ } JSON ▾ ▶ Preview 🖼 Visualize ▾
1  {
2    "success": true,
3    "message": "Authors fetched successfully",
4    "data": [
5      {
6        "_id": "6a4d275aa3e5716512586886",
7        "name": "F. Scott Fitzgerald",
8        "biography": "American novelist",
9        "country": "United States",
10       "dateOfBirth": "1896-09-24T00:00:00.000Z",
11       "createdAt": "2026-07-07T16:20:42.876Z",
12       "updatedAt": "2026-07-07T16:20:42.876Z",
13       "__v": 0
14     }
15   ],
16   "pagination": {
17     "page": 1,
18     "limit": 10,
19     "total": 1,
20     "totalPages": 1
21   }
22 }
```

2. Create Author

API Endpoint: /api/authors **Method:** POST **Description:** Creates a new author profile.

Request Body Example

```
JSON
{
  "name": "F. Scott Fitzgerald",
  "biography": "American novelist",
  "country": "United States",
  "dateOfBirth": "1896-09-24"
}
```

Authentication Requirement: Required (JWT)

{ } JSON ▾

▶ Preview

🖼 Visualize ▾

```
1  {
2    "success": true,
3    "message": "Author created successfully",
4    "data": {
5      "name": "F. Scott Fitzgerald",
6      "biography": "American novelist",
7      "country": "United States",
8      "dateOfBirth": "1896-09-24T00:00:00.000Z",
9      "_id": "6a4d275aa3e5716512586886",
10     "createdAt": "2026-07-07T16:20:42.876Z",
11     "updatedAt": "2026-07-07T16:20:42.876Z",
12     "__v": 0
13   }
14 }
```

API Module Name: Users

Endpoints Overview

Endpoint	Method	Description
/api/users	GET	Retrieve all users
/api/users/:id	GET	Retrieve user by ID
/api/users/:id	PUT	Update user details
/api/users/:id	DELETE	Delete user
/api/users/upload-profile-picture	PATCH	Upload profile picture

1. Get All Users

API Endpoint: /api/users **Method:** GET **Description:** Retrieves paginated list of users.

Query Parameters

Parameter	Type	Description
page	number	Page number (default: 1)

Parameter	Type	Description
limit	number	Items per page (default: 10)
name	string	Search by name (regex)
email	string	Filter by exact email
role	string	Filter by role (user/admin)

Authentication Requirement: Required (JWT)

```

1  {
2    "success": true,
3    "message": "Users fetched successfully",
4    "data": [
5      {
6        "_id": "6a469471eeb9a1d5654c6071",
7        "name": "test",
8        "email": "test@test.com",
9        "profilePicture": null,
10       "role": "user",
11       "createdAt": "2026-07-02T16:40:17.997Z",
12       "updatedAt": "2026-07-02T16:40:17.997Z",
13       "__v": 0
14     },
15     {
16       "_id": "6a47eae009f74f3b44be60de",
17       "name": "reader",
18       "email": "reader@test.com",
19       "profilePicture": null,
20       "role": "user",
21       "createdAt": "2026-07-03T17:01:20.635Z",
22       "updatedAt": "2026-07-03T17:01:20.635Z",
23       "__v": 0
24     },
25     {
26       "_id": "6a4d2633a3e5716512586885",
27       "name": "Atul",
28       "email": "atul@mail.com",
29       "profilePicture": null,
30       "role": "user",
31       "createdAt": "2026-07-07T16:15:47.724Z",
32       "updatedAt": "2026-07-07T16:15:47.724Z",
33       "__v": 0
34     }
35   ],
36   "pagination": {
37     "page": 1,
38     "limit": 10,
39     "total": 3,
40     "totalPages": 1
41   }
42 }
```

2. Upload Profile Picture

API Endpoint: /api/users/upload-profile-picture **Method:** PATCH **Description:** Uploads a profile picture for the authenticated user.

Request Parameters

Field Name	Type	Required	Description
profilePicture	file	Yes	Image file (jpeg, png, gif, webp)

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Profile picture uploaded successfully",
4    "data": {
5      "_id": "6a4d2633a3e5716512586885",
6      "name": "Atul",
7      "email": "atul@mail.com",
8      "profilePicture": "uploads/1783443238951.png",
9      "role": "user",
10     "createdAt": "2026-07-07T16:15:47.724Z",
11     "updatedAt": "2026-07-07T16:53:58.961Z",
12     "__v": 0
13   }
14 }
```

API Module Name: Loans

Endpoints Overview

Endpoint	Method	Description
/api/loans	GET	Retrieve all loans
/api/loans/:id	GET	Retrieve loan by ID
/api/loans	POST	Create a new loan
/api/loans/:id	PUT	Update loan details
/api/loans/:id	DELETE	Mark loan as returned

1. Get All Loans

API Endpoint: /api/loans **Method:** GET **Description:** Retrieves paginated list of loans.

Query Parameters

Parameter	Type	Description
page	number	Page number (default: 1)
limit	number	Items per page (default: 10)
status	string	Filter by status (active/returned/overdue)
user	string	Filter by user ObjectId
book	string	Filter by book ObjectId

Pagination and Filters: status, user, book **Authentication Requirement:** Required (JWT)

```
1  {
2    "success": true,
3    "message": "Loans fetched successfully",
4    "data": [
5      {
6        "_id": "6a4d2f5ec66cf66b0a2c58fb",
7        "user": {
8          "_id": "6a4d2633a3e5716512586885",
9          "name": "Atul",
10         "email": "atul@mail.com"
11       },
12       "book": {
13         "_id": "6a4d2d70fffad565265c4245",
14         "title": "The Great Gatsby"
15       },
16       "dueDate": "2026-07-21T16:54:54.853Z",
17       "returnedDate": null,
18       "status": "active",
19       "loanDate": "2026-07-07T16:54:54.854Z",
20       "createdAt": "2026-07-07T16:54:54.854Z",
21       "updatedAt": "2026-07-07T16:54:54.854Z",
22       "__v": 0
23     },
24   ],
25   "pagination": {
26     "page": 1,
27     "limit": 10,
28     "total": 1,
29     "totalPages": 1
30   }
31 }
```

2. Create Loan

API Endpoint: /api/loans **Method:** POST **Description:** Creates a loan and decrements available copies.

Request Body Example

JSON

```
{ "book": "664a1b2c3d4e5f6a7b8c9d0e", "dueDate": "2026-07-21" }
```

Response Example

JSON

```
{
  "success": true,
  "message": "Loan created successfully",
  "data": {
    "_id": "6a4d2f5ec66cf66b0a2c58fb",
    "user": {
      "_id": "6a4d2633a3e5716512586885",
      "name": "Atul",
      "email": "atul@mail.com"
    },
    "book": {
      "_id": "6a4d2d70fffad565265c4245",
      "title": "The Great Gatsby"
    },
    "dueDate": "2026-07-21T16:54:54.853Z",
    "returnedDate": null,
    "status": "active",
    "loanDate": "2026-07-07T16:54:54.854Z",
    "createdAt": "2026-07-07T16:54:54.854Z",
    "updatedAt": "2026-07-07T16:54:54.854Z",
    "__v": 0
  }
}
```

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Loan created successfully",
4    "data": {
5      "_id": "6a4d2f5ec66cf66b0a2c58fb",
6      "user": {
7        "_id": "6a4d2633a3e5716512586885",
8        "name": "Atul",
9        "email": "atul@mail.com"
10     },
11     "book": {
12       "_id": "6a4d2d70ffad565265c4245",
13       "title": "The Great Gatsby"
14     },
15     "dueDate": "2026-07-21T16:54:54.853Z",
16     "returnedDate": null,
17     "status": "active",
18     "loanDate": "2026-07-07T16:54:54.854Z",
19     "createdAt": "2026-07-07T16:54:54.854Z",
20     "updatedAt": "2026-07-07T16:54:54.854Z",
21     "__v": 0
22   }
23 }
```

3. Return Loan (DELETE)

API Endpoint: /api/loans/:id **Method:** DELETE **Description:** Marks loan as returned, sets returnedDate, increments availableCopies.

Response Example

JSON

```
{ "success": true, "message": "Loan marked as returned successfully" }
```

Behavior: Returns book, sets date, increments copies **Authentication Requirement:** Required (JWT)

```
1  {
2    "success": true,
3    "message": "Loan marked as returned successfully"
4  }
```

API Module Name: Reviews

Endpoints Overview

Endpoint	Method	Description
/api/books/:id/reviews	GET	Get reviews for a book
/api/books/:id/reviews	POST	Add a review
/api/books/:id/reviews/:reviewId	PUT	Update a review
/api/books/:id/reviews/:reviewId	DELETE	Delete a review

1. Get Book Reviews

API Endpoint: /api/books/:id/reviews **Method:** GET **Description:** Retrieves paginated reviews for a specific book.

Query Parameters

Parameter	Type	Description
page	number	Page number (default: 1)
limit	number	Items per page (default: 10)

Authentication Requirement: None

```
1  {
2    "success": true,
3    "message": "Reviews fetched successfully",
4    "data": [
5      {
6        "_id": "6a4d2ff7c66cf66b0a2c58fc",
7        "user": {
8          "_id": "6a4d2633a3e5716512586885",
9          "name": "Atul",
10         "email": "atul@mail.com"
11       },
12       "book": "6a4d2d70ffad565265c4245",
13       "rating": 4,
14       "review": "Great book, highly recommend!",
15       "createdAt": "2026-07-07T16:57:27.400Z",
16       "updatedAt": "2026-07-07T16:57:27.400Z",
17       "__v": 0
18     },
19   ],
20   "pagination": {
21     "page": 1,
22     "limit": 10,
23     "total": 1,
24     "totalPages": 1
25   }
26 }
```

2. Create Review

API Endpoint: /api/books/:id/reviews **Method:** POST **Description:** Adds a review for a book (one review per user per book).

Request Parameters

Field Name	Type	Required	Description	Allowed Values
rating	number	Yes	Book rating	1-5
review	string	No	Review text	N/A

Request Body Example

JSON

```
{ "rating": 4, "review": "Great book, highly recommend!" }
```

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Review created successfully",
4    "data": {
5      "_id": "6a4d2ff7c66cf66b0a2c58fc",
6      "user": {
7        "_id": "6a4d2633a3e5716512586885",
8        "name": "Atul",
9        "email": "atul@mail.com"
10     },
11     "book": "6a4d2d70ffad565265c4245",
12     "rating": 4,
13     "review": "Great book, highly recommend!",
14     "createdAt": "2026-07-07T16:57:27.400Z",
15     "updatedAt": "2026-07-07T16:57:27.400Z",
16     "__v": 0
17   }
18 }
```

3. Update Review

API Endpoint: /api/books/:id/reviews/:reviewId **Method:** PUT **Description:** Updates a review. Only the review author can update.

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Review updated successfully",
4    "data": {
5      "_id": "6a4d2ff7c66cf66b0a2c58fc",
6      "user": {
7        "_id": "6a4d2633a3e5716512586885",
8        "name": "Atul",
9        "email": "atul@mail.com"
10     },
11     "book": "6a4d2d70ffad565265c4245",
12     "rating": 5,
13     "review": "Great book, highly recommend!",
14     "createdAt": "2026-07-07T16:57:27.400Z",
15     "updatedAt": "2026-07-07T16:58:47.460Z",
16     "__v": 0
17   }
18 }
```

4. Delete Review

API Endpoint: /api/books/:id/reviews/:reviewId **Method:** DELETE **Description:** Deletes a review. Only the review author can delete.

Authentication Requirement: Required (JWT)

```
1  {
2    "success": true,
3    "message": "Review deleted successfully"
4  }
```